

Linux & Cyber Forensics Student Practice Lab

Environment: Kali Linux / Ubuntu

Level: Beginner → Intermediate

Objective: Practice the commands used in class through practical scenarios.

Important: Perform these exercises only on your own VM/lab system. Do not modify another user's account, logs, or system files without authorization.

Basic File & Directory Operations

Create a working directory:

```
mkdir -p ~/student-lab/{evidence,working,reports,backup}  
cd ~/student-lab
```

Tasks:

1. Display your current directory.
2. List all files, including hidden files.
3. Create a file called `notes.txt`.
4. Create three directories: `logs`, `samples`, and `output`.
5. Create a file called `evidence.txt`.
6. Copy `evidence.txt` into `backup/`.
7. Rename `notes.txt` to `investigator_notes.txt`.
8. Remove an empty directory.
9. Display the directory structure using `tree`.

Commands students should practice:

```
pwd  
ls  
ls -la  
mkdir  
touch  
cp  
mv  
rmdir  
tree
```

Creating and Viewing Evidence

Create a sample evidence file:

```
cd ~/student-lab/evidence
nano evidence.txt
```

Put several lines such as:

```
User: analyst1
Status: failed
IP: 192.168.1.10
Event: login
Status: successful
IP: 192.168.1.20
Event: file_access
Status: failed
IP: 192.168.1.10
Event: login
```

Practice:

```
cat evidence.txt
less evidence.txt
more evidence.txt
head -5 evidence.txt
tail -5 evidence.txt
```

Questions

- What is the difference between `cat` and `less`?
- What does `head` display?
- What does `tail` display?

grep Investigation

Using `evidence.txt`, find:

Task 1

Find all lines containing `failed`.

```
grep 'failed' evidence.txt
```

Task 2

Perform a case-insensitive search for `FAILED`.

```
grep -i 'failed' evidence.txt
```

Task 3

Display line numbers.

```
grep -n 'failed' evidence.txt
```

Task 4

Find lines containing an IP address.

```
grep -E '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' evidence.txt
```

Task 5

Count failed events.

```
grep 'failed' evidence.txt | wc -l
```

This is especially useful for SOC/log-analysis practice. Your class history used the same `grep → pipe → wc` pattern for counting log events.

Pipes |

Students must understand:

Command 1 → output → Command 2

Practice:

```
ls | wc -l
```

Then:

```
cat evidence.txt | grep 'failed'
```

Then:

```
cat evidence.txt | grep 'failed' | wc -l
```

Challenge

Find how many lines contain an IP address.

Redirection > and >>

Practice:

```
ls > file_list.txt
```

View it:

```
cat file_list.txt
```

Now append:

```
ls >> file_list.txt
```

View again:

```
cat file_list.txt
```

Students should explain:

Operator	Purpose
>	Overwrites/creates output file
>>	Appends to output file
	Sends output to another command

Your class history specifically practiced `ls > list.txt` and `ls >> list.txt`.

sort and uniq

Create:

```
nano users.txt
```

Add:

```
admin  
analyst  
admin  
guest  
analyst
```

```
admin
guest
```

Run:

```
sort users.txt
```

Then:

```
sort users.txt | uniq
```

Then:

```
sort users.txt | uniq -c
```

Finally:

```
sort users.txt | uniq -c | sort -rn
```

Challenge

Which username appears most frequently?

This reproduces the log-analysis pattern practiced in class.

cut

Practice with `/etc/passwd`:

```
cat /etc/passwd
```

Extract usernames:

```
cut -d ':' -f1 /etc/passwd
```

Extract another field:

```
cut -d ':' -f5 /etc/passwd
```

Questions

1. What does `-d ':'` mean?
2. What does `-f1` mean?
3. Why is `/etc/passwd` useful during Linux forensic investigation?

The class history contains several exercises using `cut` against `/etc/passwd`.

tr

Convert lowercase to uppercase:

```
echo 'hello security' | tr 'a-z' 'A-Z'
```

Convert uppercase to lowercase:

```
echo 'HELLO SECURITY' | tr 'A-Z' 'a-z'
```

Challenge

Convert:

```
Cyber Security Forensics
```

to uppercase using `tr`.

Log Analysis with `awk`

Create an `access.log` containing lines similar to:

```
2026-08-18 09:01:12 192.168.1.10 admin 200 GET /login
2026-08-18 09:01:20 192.168.1.10 admin 401 POST /login
2026-08-18 09:01:21 192.168.1.10 admin 401 POST /login
2026-08-18 09:02:10 192.168.1.20 guest 404 GET /admin
2026-08-18 09:03:15 192.168.1.30 analyst 200 GET /dashboard
2026-08-18 09:04:22 192.168.1.10 admin 401 POST /login
```

Your class practiced extracting fields from this type of log with `awk`.

Tasks

Display the IP:

```
awk '{print $3}' access.log
```

Display IP and username:

```
awk '{print $3, $4}' access.log
```

Find HTTP 401 events:

```
awk '$5 == 401 {print $3, $4}' access.log
```

Display IP, username, and requested resource for 403:

```
awk '$5 == 403 {print $3, $4, $7}' access.log
```

Challenge

Find the IP generating the most 401 responses:

```
awk '$5 == 401 {print $3}' access.log | sort | uniq -c | sort -rn
```

This is an excellent **SOC L1 brute-force investigation exercise**.

sed

Display the first five lines:

```
sed -n '1,5p' access.log
```

Display lines containing /login:

```
sed -n '/\/login/p' access.log
```

Display lines containing 404:

```
sed -n '/404/p' access.log
```

Remove lines containing 500 from the displayed output:

```
sed '/500/d' access.log
```

Remove comment lines:

```
sed '/^#/d' access.log
```

Your original practice history contains these exact `sed` investigation patterns.

— **xxd** and **strings**

Create a sample file:

```
echo "This is a forensic sample" > sample.txt
```

View hexadecimal representation:

```
xxd sample.txt
```

Extract readable strings:

```
strings sample.txt
```

Students should understand:

```
xxd      → hexadecimal representation  
strings → printable/readable strings
```

These were also part of your class practice.

File Discovery

Find all .txt files:

```
find . -name "*.txt"
```

Find directories:

```
find . -type d
```

Forensics challenge

Find SUID files:

```
find / -perm -4000 -type f 2>/dev/null
```

Explain why SUID files are interesting during a **Linux security/forensic investigation**.

Your class practiced this SUID discovery command.

Linux Users and Groups

First inspect your current identity:

```
whoami  
id
```

View users:

```
cat /etc/passwd
```

View groups:

```
cat /etc/group
```

Create a lab user:

```
sudo useradd -m analyst1
```

Check:

```
id analyst1
```

Create a group:

```
sudo groupadd security
```

Add the user:

```
sudo gpasswd -a analyst1 security
```

Verify:

```
groups analyst1
```

Your class history included `useradd`, `usermod`, `groupadd`, `gpasswd`, `groups`, and `id`.

Mini SOC Investigation

Give students the following scenario:

Scenario:

You are a SOC analyst. The web server has reported multiple failed login attempts. Your job is to identify the suspicious IP address and determine how many failed authentication attempts it generated.

Students receive `access.log`.

They must answer:

1. How many total log entries exist?

2. Which entries contain 401?
3. Which IP addresses generated 401 responses?
4. Which IP generated the most 401 responses?
5. Which username was targeted?
6. Which URL was accessed?
7. Are there any 404 requests?
8. Are there any 500 errors?
9. Produce a short investigation summary.

Recommended investigation pipeline

```
grep '401' access.log  
awk '$5 == 401 {print $3}' access.log  
awk '$5 == 401 {print $3}' access.log | sort | uniq -c | sort -rn
```

The final command is directly aligned with the investigation pattern you practiced in class.

Final Student Challenge

Don't give students the commands below. Give them **only the objectives**:

Investigation Challenge

You have been provided with `access.log`.

Without searching online, use Linux command-line tools to:

- Identify all IP addresses.
- Count the number of unique IP addresses.
- Find all failed authentication attempts.
- Identify the most frequent attacking IP.
- Identify the targeted username.
- Find all HTTP 404 responses.
- Find all HTTP 500 responses.
- Count login requests.
- Extract the requested URLs.
- Sort the most frequent IP addresses.
- Save your investigation results into `report.txt`.

This forces students to combine:

grep → awk → cut → sort → uniq → wc → sed → pipes → redirection

rather than memorizing individual commands.

Commands covered

Area	Commands
Navigation	pwd, cd, ls
Files	touch, cp, mv, rm
Directories	mkdir, rmdir, tree
Viewing	cat, less, more, head, tail
Search	grep, find
Text processing	cut, tr, sort, uniq
Log analysis	awk, sed
Output	>, >>,
Forensics	sha256sum, file, stat, xxd, strings
Users	useradd, usermod, userdel, id, whoami
Groups	groupadd, gpasswd, groups
Permissions	chmod
System investigation	ss, lsof, ps

I would make **this the student handout**, and keep a separate **instructor answer key** containing the commands/expected results. That way students actually practice instead of copying the command history.